



MALATYA TURGUT ÖZAL ÜNİVERSİTESİ
SOSYAL VE BEŞERİ BİLİMLER FAKÜLTESİ
YÖNETİM BİLİŞİM SİSTEMLERİ BÖLÜMÜ

,

NESNE TABANLI PROGRAMLAMA-II
11. HAFTA (23 NİSAN- 30 NİSAN 2026) UYGULAMA FÖYÜ

ÖĞRETİM ÜYESİ: DR. ÖĞR. ÜYESİ SEDA İŞGÜZAR

UYGULAMA-1:

Bir havayolu şirketinin rezervasyon sistemini modelleyen, grafik arayüz (Swing) tabanlı bir Java uygulaması geliştirmeniz beklenmektedir. Sistem, veritabanı kullanmadan, uçuş bilgilerini dizi veya koleksiyon yapıları içerisinde saklayacaktır. Uygulamada yolcu adına rezervasyon yapma, rezervasyon iptal etme ve iki uçuş arasında aktarmalı rezervasyon gerçekleştirme işlemleri yer almalıdır.

Sistem içerisinde aşağıdaki hata durumları söz konusudur ve bu durumların tamamı özel (custom) checked exception sınıfları ile modellenmelidir:

Uçuşun iptal edilmiş olması → `UcusIptalException`

Kalkışa 30 dakikadan az süre kalmış olması → `KalkisSuresiGecmisException`

Uçuş kapasitesinin dolu olması → `UcusDoluException`

Yolcunun pasaportunun geçersiz olması → `GecersizPasaportException`

Bu istisna sınıfları, `RezervasyonException` adlı üst sınıftan türetilmelidir.

RezervasyonException	üst sınıf (checked)
└─ UcusDoluException	kapasite aşıldı
└─ KalkisSuresiGecmisException	kalkışa < 30 dk kaldı
└─ UcusIptalException	uçuş zaten iptal
└─ GecersizPasaportException	yolcu pasaportu geçersiz

Uygulama aşağıdaki gereksinimleri sağlamalıdır:

- Swing kullanılarak bir kullanıcı arayüzü oluşturulmalıdır. Arayüzde:
 - Uçuşların listelendiği bir bileşen (örneğin `JList`)
 - Yolcu adı girişi için metin alanı
 - Pasaport geçerliliğini temsil eden bir seçim bileşeni (örneğin `JCheckBox`)
 - “Rezervasyon Yap”, “İptal Et” ve “Aktarma Yap” butonları bulunmalıdır.
- “Rezervasyon Yap” işlemi sırasında:
 - Yukarıda belirtilen hata durumları kontrol edilmeli,
 - Uygun durumda ilgili özel exception fırlatılmalıdır.
- “Aktarma Yap” işlemi sırasında:
 - İlk uçuş başarısız olursa ikinci uçuş kesinlikle denenmemelidir.
 - Hata durumları exception chaining (cause) kullanılarak zincir halinde raporlanmalıdır.
- Tüm exception türleri, main veya event handling içerisinde ayrı ayrı catch bloklarında ele alınmalıdır.
- Her işlem sonrası (başarılı veya hatalı), sistem:
 - Toplam rezervasyon deneme sayısını
 - Başarılı rezervasyon sayısını
 - Başarısız rezervasyon sayısınıarayüz başlığı veya uygun bir bileşende göstermelidir.

6. Exception zincirini yazdırmak için getCause() metodunu kullanan yardımcı bir metod yazılmalı ve hata durumunda kullanıcıya detaylı bilgi sunulmalıdır.
7. Tüm işlemlerde finally bloğu kullanılarak arayüzün güncellenmesi garanti altına alınmalıdır.

Kodunuz nesne tabanlı programlama prensiplerine uygun olmalı, sınıf yapısı açık ve düzenli şekilde tasarlanmalıdır. Basit arayüz yapısı aşağıda verilmiştir.

Deneme:1 | Başarılı:1

Yolcu: ☐ Pasaport Geçerli

Rezervasyon İptal Aktarma

Program arayüzünü derleyince Şekil-2’de ekran gelir. Yolcu adı yazılıp pasaport geçerliliğine dair checkbox işaretlenir. Açık olan rezervasyon üzerine gelinir ve rezervasyon butonuna tıklanır.

Uçuş Sistemi

Yolcu: seda işgüzar

☒ Pasaport Geçerli

TK101 | Kalan:0 | dk:120

TK202 | Kalan:2 | dk:20

TK303 | Kalan:5 | dk:200 [İPTAL]

Rezervasyon

İptal

Aktarma

Uçuş Sistemi

Yolcu: seda işgüzar

☒ Pasaport Geçerli

TK101 | Kalan:0 | dk:120

TK202 | Kalan:2 | dk:20

TK303 | Kalan:5 | dk:200 [İPTAL]

Rezervasyon

İptal

Aktarma

Rezervasyon yapılmaya çalışıldığında Eğer yer yoksa Uçuş Dolu, Uçuşun Kalkış saati geçmişse Kalkış Süresi Geçti, Uçuş iptal edilmişse İptal edildi mesajları dönmelidir. Büyün bunlar üreteğiniz istisna sınıfları yönetilmelidir.

Uçuş Sistemi

Yolcu: seda işgüzar

☒ Pasaport Geçerli

TK101 | Kalan:0 | dk:120

TK202 | Kalan:2 | dk:20

TK303 | Kalan:5 | dk:200 [İPTAL]

Message



Uçuş dolu

OK

Rezervasyon

İptal

Aktarma

